RACING

R-based Attractor Complexity INdex for Gait

User Guide

Complete Function Reference and Pipeline Documentation

Version 1.0 — March 2026

ORD 2025 Project

HE-Arc Santé Neuchâtel

Philippe Terrier

HE-Arc Santé, HES-SO University of Applied Sciences and Arts Western Switzerland

Table of Contents

Preface

This User Guide provides comprehensive documentation for the RACING (R-based Attractor Complexity INdex for Gait) software package. RACING is an open-source R implementation of validated MATLAB algorithms for linear and nonlinear gait analysis using trunk-worn accelerometer data. The package was developed as part of the ORD 2025 (Open Research Data) project at HE-Arc Santé Neuchâtel, with funding from the HES-SO University of Applied Sciences and Arts Western Switzerland.

The algorithms implemented in RACING were originally developed and validated in the ACIER study (Attractor Complexity Index Empirical Rationalization, 2021–2024), which analyzed gait quality in older adults using triaxial accelerometer data from a lower-back sensor. The MATLAB implementations were published alongside two peer-reviewed articles in Scientific Reports and Sensors. The RACING project makes these algorithms freely available to the international research community by porting them to the open-source R language, following FAIR (Findable, Accessible, Interoperable, Reusable) principles.

How to Use This Guide

This guide is organized following the data processing pipeline, from raw accelerometer data to final gait quality metrics. Each chapter corresponds to one or more R scripts and provides the scientific background, function signatures, parameter descriptions, usage examples, error handling, and integration notes.

Chapter 1 introduces the scientific context and project background. **Chapter 2** describes the pipeline architecture and processing order. **Chapters 3–10** document individual analysis scripts in pipeline order. **Chapter 11** covers the batch processing system that orchestrates the full pipeline. The **Appendices** provide a quick reference card, glossary of terms, and complete bibliography.

Conventions

Throughout this guide, R code is displayed in `monospaced font`. Function names are written as `function_name()`. File names are shown as *`script_name.R`*. Default parameter values appear in square brackets, e.g., [256]. Cross-references to other chapters are indicated by chapter number.

Software Requirements

RACING requires R version 4.0 or later. The following packages are needed:

- **gsignal** — MATLAB-compatible signal resampling (polyphase filtering)
- **RANN** — Fast nearest-neighbor search via KD-trees (strongly recommended for performance)
- **data.table** — Fast CSV import (optional but recommended)
- **readxl** — Reading Excel configuration files
- **openxlsx** — Writing Excel output files

Install all dependencies with:

```
install.packages(c("gsignal", "RANN", "data.table", "readxl", "openxlsx"))
```

1. Introduction

1.1 Scientific Context

Falls in older adults represent a major public health challenge. Approximately one-third of adults over 65 experience at least one fall per year, often with serious consequences including fractures, loss of autonomy, and increased mortality. Early detection of gait instability could enable preventive interventions before falls occur.

Nonlinear dynamical analysis of trunk acceleration during walking offers a promising approach to quantifying gait quality beyond traditional clinical measures. By reconstructing the phase space attractor from a single accelerometer signal, researchers can assess both local dynamic stability (sensitivity to perturbations within a gait cycle) and attractor complexity (the structure of stride-to-stride fluctuations). These measures have been shown to distinguish between fallers and non-fallers, between younger and older adults, and between different walking conditions.

1.2 The ACIER Study

The ACIER study (Attractor Complexity Index Empirical Rationalization) was conducted at HE-Arc Santé Neuchâtel between 2021 and 2024. It collected triaxial accelerometer data from 102 participants (older and younger adults) wearing a Physilog sensor on the lower back at 256 Hz. Participants walked under multiple conditions: normal indoor, metronome-paced indoor, and normal outdoor. The study produced two publications:

- Piergiovanni & Terrier (2024), Scientific Reports: Effects of metronome walking on long-term attractor divergence and correlation structure of gait.
- Piergiovanni & Terrier (2024), Sensors: Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer.

The raw accelerometer data are publicly available on Zenodo (DOI: 10.5281/zenodo.10148824).

1.3 The RACING Project

RACING (R-based Attractor Complexity INdex for Gait) is an 8-month ORD 2025 project that ports the validated MATLAB algorithms from the ACIER study to R, making them open-source and accessible to researchers worldwide. The project follows FAIR principles: the source code is hosted on GitHub, the software is described in a SoftwareX article, and a CodeOcean reproducibility capsule provides a citable, executable demonstration.

The R implementation was developed with a validation-first approach: each function was verified against the original MATLAB code line-by-line, and cross-platform numerical agreement was confirmed on the full ACIER dataset. The pipeline achieves exact agreement for linear metrics (step frequency, RMS, ACF regularity) and near-perfect agreement for nonlinear metrics, with a documented systematic offset of approximately 0.008 in ACI values attributable to differences in polyphase resampling filter implementations between MATLAB and R.

1.4 Gait Quality Metrics

RACING computes eight categories of gait metrics from a single lower-back accelerometer recording:

Metric	Abbreviation	Interpretation
Step frequency	SF	Cadence in Hz (steps per second)
RMS movement intensity	RMS_norm	Average acceleration magnitude; proxy for walking speed
RMS lateral ratio	RMS_ratio	Mediolateral sway relative to total movement; balance index
Step regularity	step_reg	Consistency of left-right alternation (ACF-based, Fisher z)
Stride regularity	stride_reg	Consistency of full gait cycles (ACF-based, Fisher z)
Average Mutual Information	AMI	Optimal embedding delay for phase space reconstruction
Local Dynamic Stability	LDS	Short-term divergence rate; dynamic balance measure
Attractor Complexity Index	ACI	Long-term divergence slope; gait automaticity measure

2. Pipeline Architecture

2.1 Processing Order

The RACING pipeline processes each accelerometer file through a fixed sequence of operations. The processing order is critical and must not be altered, as several steps depend on outputs from previous steps:

Step	Operation	Script	Signal Version
0	Compute vector norm	(inline)	Raw signals
1	Tilt correction	tilt_correction.R	Raw signals
2	Step frequency detection	step_frequency_detection.R	Corrected vertical
3	SF rounding to 0.1 Hz	(inline)	N/A
4	Signal truncation	truncate_resample.R	All axes + norm
5	Signal resampling	resample_signal.R	All axes + norm
6	RMS computation	compute_rms.R	Truncated
7	ACF gait regularity	acf_gait_regularity.R	Truncated norm
8	AMI (4 axes)	ami.R	Resampled
9	Rosenstein divergence (4 axes)	rosenstein_divergence.R	Resampled
10	LDS/ACI fitting (4 axes)	fit_divergence_exponents.R	Resampled

2.2 Dual Signal Architecture

A key design principle is the production of two distinct signal versions from the preprocessing stage:

Truncated signals are cut to a standardized number of steps at the original sampling rate (e.g., 256 Hz). These preserve actual temporal relationships and are used for linear metrics: RMS movement intensity, step frequency, and ACF-based step/stride regularity.

Resampled signals are further processed to a fixed length (default: 18,750 samples = 250 steps at 75 samples/step). This standardization ensures every subject has the same number of samples per step, which is required for valid phase space reconstruction and inter-subject comparison of nonlinear metrics (AMI, LDS, ACI).

2.3 Script Layers

The 13 R scripts are organized in three functional layers:

Layer 1 — Standalone metrics: tilt_correction.R, step_frequency_detection.R, acf_gait_regularity.R, ami.R, rosenstein_divergence.R, fit_divergence_exponents.R, compute_rms.R, resample_signal.R. Each script is self-contained and can be used independently.

Layer 2 — Preprocessing orchestration: truncate_resample.R integrates tilt correction, step frequency detection, and resampling into a single call that produces the dual signal outputs.

Layer 3 — Batch processing: read_config.R, import_csv_lowback.R, run_gait_analysis_batch.R, and write_results.R handle configuration, data import, pipeline orchestration, and output formatting for processing entire datasets.

2.4 Configuration System

All analysis parameters are stored in a RACING configuration Excel file (RACING_config.xlsx). The configuration sheet contains rows for input data paths, preprocessing options, signal processing constants, nonlinear analysis parameters, and output settings. This centralized design ensures reproducibility: the same configuration file applied to the same data will always produce identical results.

Essential default constants:

```
sampling_freq     <- 256     # Hz (Physilog sensor)
target_steps      <- 250     # Standardized step count
samples_per_step  <- 75      # After resampling
target_samples    <- 18750   # 250 x 75
embedding_dim     <- 5       # Phase space dimensions
lds_range         <- c(0, 75)     # Samples (0-0.5 strides)
aci_range         <- c(750, 1800) # Samples (5-12 strides)
```

3. Tilt Correction (tilt_correction.R)

3.1 Purpose

This script implements the Moe-Nilssen (1998) tilt correction algorithm, which transforms triaxial accelerometer data from an arbitrarily tilted sensor frame into a horizontal-vertical earth frame, removing the effect of gravity. The corrected signals feed into step frequency detection (performed on the corrected vertical axis) and all subsequent analyses.

3.2 Scientific Background

When a triaxial accelerometer is attached to the lower back during walking, it is rarely perfectly aligned with the anatomical axes. Even small sensor tilts cause gravity (1 g) to project onto the horizontal axes, creating a static offset that contaminates the anteroposterior (AP) and mediolateral (ML) acceleration signals. For a typical forward tilt of 10 degrees, the AP axis registers a static 0.17 g offset.

The algorithm exploits the fact that over a complete walking trial, the mean dynamic acceleration on each axis approaches zero (cyclical movement). Therefore, the measured mean acceleration equals the gravity component. Two sequential 2D rotations transform the data back to the earth frame.

Rotation Equations (A1–A4)

Eq.	Formula	Description
A1	a_A = a_ap*cos(theta_a) - a_v*sin(theta_a)	Corrected AP (horizontal)
A2	a_v' = a_ap*sin(theta_a) + a_v*cos(theta_a)	Provisional vertical
A3	a_M = a_ml*cos(theta_m) - a_v'*sin(theta_m)	Corrected ML (horizontal)
A4	a_V = a_ml*sin(theta_m) + a_v'*cos(theta_m) - 1	Final vertical (gravity removed)

The −1 in Equation A4 removes the static 1 g gravity component so that the corrected vertical acceleration has a mean near zero.

3.3 Function Reference

tilt_correction()

Main function. Applies two sequential 2D rotations and removes gravity.

Parameter	Type	Default	Description
acc_ap	numeric	(required)	AP acceleration vector in g units
acc_v	numeric	(required)	Vertical acceleration vector in g units
acc_ml	numeric	(required)	ML acceleration vector in g units
auto_orient	logical	TRUE	Auto-detect and correct inverted sensor mounting
return_diagnostics	logical	FALSE	Return tilt angles, orientation, quality check
verbose	logical	FALSE	Print diagnostic messages to console

Returns a list with corrected signals (ap, v, ml) and optionally diagnostic fields (tilt angles, orientation detection, quality check).

tilt_correction_batch()

Convenience wrapper for data frames and matrices. Extracts AP, V, and ML columns by name or index, applies tilt_correction(), and returns a data frame with columns ap_corrected, v_corrected, and ml_corrected.

validate_tilt_correction()

Post-hoc quality check. Verifies that output means are within tolerance (AP/ML: 0.05 g, V: 0.10 g) and that tilt angles are within reasonable range (default: 30 degrees).

3.4 Usage Example

```
source("tilt_correction.R")

result <- tilt_correction(
  acc_ap = raw_x, acc_v = raw_y, acc_ml = raw_z,
  return_diagnostics = TRUE, verbose = TRUE
)

corrected_x <- result$ap    # AP, mean ~ 0
corrected_y <- result$v     # V, gravity removed
corrected_z <- result$ml    # ML, mean ~ 0

validation <- validate_tilt_correction(result)
if (!validation$valid) warning(validation$messages)
```

3.5 Methodological Notes

Orientation detection. The R implementation adds automatic orientation detection not present in the original paper. When mean(V) < −0.1 g, the vertical signal is flipped before applying the rotation equations. The threshold of −0.1 g reliably distinguishes normal mounting (mean V ~ +0.98 g) from inverted mounting (mean V ~ −0.98 g).

Rotation order. The AP correction must precede the ML correction because the second rotation uses the provisional vertical from the first. Reversing the order gives incorrect results. For tilts up to approximately 28 degrees (typical gait range), the sequential rotation approximation is excellent.

Yaw limitation. The algorithm cannot correct for yaw rotation (rotation about the vertical axis). This is acceptable for controlled laboratory settings but may be problematic for free-living monitoring where the sensor rotates during extended wearing.

3.6 Error Handling

Condition	Response
Non-numeric inputs	stop() with descriptive message

Vectors of different length	stop() with descriptive message
Fewer than 10 samples	warning(); angle estimate unreliable
NA values in input	warning(); propagated to output
|sin(theta)| > 1.0	Clamped to 0.999
|sin(theta)| > 1.1	warning(); likely wrong units
Output means not near 0	quality_check = FALSE in diagnostics

4. Step Frequency Detection (step_frequency_detection.R)

4.1 Purpose

This script estimates step frequency from vertical trunk acceleration using FFT power spectral density analysis with parabolic interpolation for sub-bin frequency precision. The detected step frequency serves two purposes: scientific reporting of cadence, and computing the truncation index for extracting a standardized number of steps.

4.2 Scientific Background

Step frequency is the rate of stepping during walking, typically 1.5–2.3 Hz (90–138 steps/min) for adults. The R implementation adds parabolic (quadratic) interpolation on the log-magnitude spectrum to achieve sub-bin frequency precision (Gasior & Gonzalez, 2004). This provides robustness against the FFT bin shifts (~0.004 Hz) that can arise from floating-point differences in tilt correction between MATLAB and R.

SF Rounding Mechanism

To ensure MATLAB/R convergence for truncation, both implementations round SF to 0.1 Hz before computing the truncation index. The precise SF is preserved separately for scientific reporting. The truncation formula is: floor(sampling_freq / round(SF, 1) * target_steps).

4.3 Function Reference

step_frequency_fft()

Main analysis function. Computes step frequency via FFT with parabolic interpolation.

Parameter	Type	Default	Description
acceleration_vertical	numeric	(required)	Vertical acceleration vector in g
sampling_freq	numeric	256	Sampling frequency in Hz
freq_range	numeric[2]	c(1.4, 3.0)	Min/max frequency search range (Hz)
detrend	logical	TRUE	Remove signal mean before FFT
zero_pad	logical	FALSE	Zero-pad to next power of 2
return_spectrum	logical	FALSE	Include PSD vectors in output

Returns a list containing: step_freq_hz (interpolated frequency), step_freq_bpm, peak_bin_freq, peak_bin_index, interp_delta, peak_power, freq_resolution, method, quality sub-list, and optionally psd/frequencies vectors.

plot_step_frequency()

Diagnostic two-panel visualization. Top panel shows the time-domain signal; bottom panel shows the PSD with annotated search range, discrete peak, and interpolated frequency. Requires return_spectrum = TRUE in the preceding call.

step_frequency()

Convenience wrapper that forwards all arguments to step_frequency_fft().

4.4 Usage Example

```
source("step_frequency_detection.R")

sf <- step_frequency_fft(acc_y_corrected,
  sampling_freq = 256, return_spectrum = TRUE)

cat(sprintf("SF = %.4f Hz (%.1f bpm)\n",
  sf$step_freq_hz, sf$step_freq_bpm))

# Diagnostic plot:
plot_step_frequency(acc_y_corrected, sf)
```

4.5 Error Handling

Condition	Response
Signal < 100 samples	stop()
Signal < 4 seconds	warning(); reduced resolution
< 3 bins in freq_range	stop(); range too narrow
Peak at spectrum edge	warning; interpolation skipped
Zero magnitude neighbor	warning; interpolation skipped
Non-concave peak shape	warning; interpolation skipped
|delta| > 0.5	Clamped to +/-0.5 with warning

5. Preprocessing Orchestration (truncate_resample.R)

5.1 Purpose

This is the central preprocessing orchestrator of the RACING pipeline. It transforms raw triaxial accelerometer signals into two analysis-ready signal sets: truncated signals at the original sampling rate for linear metrics, and resampled signals at a standardized length for nonlinear dynamics analysis. The function integrates tilt correction, step frequency detection, and bandlimited resampling into a single call that replicates the exact processing order of the MATLAB ACIER pipeline.

5.2 Processing Steps

Step	Operation	Notes
0	Compute vector norm from raw signals	sqrt(x^2+y^2+z^2) - 1, before correction
1	Tilt correction (Moe-Nilssen)	Optional; auto-orient supported
2	SF detection (FFT on corrected V)	Always internal when tilt ON
3	Round SF to 0.1 Hz	Ensures MATLAB/R convergence
4	Compute truncation index	floor(fs / SF * N_steps)
5	Truncate all axes + norm	signal[1:truncation_index]
6	Compute target length	N_steps x samples_per_step
7	Resample (bandlimited sinc)	Auto two-stage for ratio near 1
8	Return S3 gait_preprocessed object	truncated + resampled + metadata

5.3 Function Reference

truncate_resample()

Parameter	Type	Default	Description
acc_x, acc_y, acc_z	numeric	(required)	Triaxial acceleration in g
sampling_freq	numeric	256	Sampling frequency in Hz
target_steps	integer	250	Number of steps to extract
target_samples	integer/NULL	NULL	Target resampled length (auto-computed if NULL)
samples_per_step	integer	75	Samples per step after resampling
apply_tilt_correction	logical	TRUE	Apply Moe-Nilssen correction
auto_orient	logical	TRUE	Auto-detect inverted mounting

step_freq_hz	numeric/NULL	NULL	Pre-computed SF (IGNORED when tilt ON)
freq_range	numeric(2)	c(1.2, 3.0)	Min/max Hz for SF detection
resampling_method	character	"auto"	"auto", "direct", or "two_stage"
verbose	logical	TRUE	Print progress messages
log_file	character/NULL	NULL	Path to debug log for resampling

Return Value (S3 gait_preprocessed object)

Component	Contents
truncated	Truncated signals at original rate: x, y, z, norm
resampled	Resampled signals at standardized length: x, y, z, norm
metadata	step_freq_hz, actual_steps, truncated_length, resampled_length, resampling_ratio, method, etc.
tilt_correction	theta_ap_deg, theta_ml_deg, orientation_detected, orientation_corrected
quality	tilt_correction_valid, warnings (character vector)

5.4 Usage Example

```
source("tilt_correction.R")
source("step_frequency_detection.R")
source("resample_signal.R")
source("truncate_resample.R")

data <- read.csv("subject_001_NC.csv")
result <- truncate_resample(
  acc_x = data$acc_x, acc_y = data$acc_y, acc_z = data$acc_z,
  sampling_freq = 256, target_steps = 250, verbose = TRUE
)
print(result)  # Formatted summary

# Downstream: linear metrics on truncated signals
rms <- compute_rms(result$truncated$z, result$truncated$norm)

# Downstream: nonlinear metrics on resampled signals
ami_result <- ami(result$resampled$norm, n_bins = 64, n_lags = 100)
```

5.5 Critical Design Decisions

SF re-detection override. When apply_tilt_correction=TRUE, any externally provided step_freq_hz is ignored and SF is re-detected on the tilt-corrected vertical signal. This prevents a subtle 1-FFT-bin shift (~0.003 Hz) that occurs when SF is detected on raw (uncorrected) data.

Norm computed before tilt correction. The vector norm is computed from raw signals before tilt correction. Mathematically, the norm is invariant to rotation, so the order does not affect the result. The specific order is preserved for reproducibility with MATLAB outputs.

Automatic problematic-ratio handling. The MATLAB code hardcoded 8 subject IDs needing a 2/3 pre-downsampling step. The R implementation detects these cases automatically via resample_signal() and applies two-stage resampling when needed, generalizing to any dataset.

6. Signal Resampling (resample_signal.R)

6.1 Purpose

This script resamples truncated gait acceleration signals to a standardized length using polyphase filtering, replicating MATLAB's resample(x, p, q, N, beta) algorithm. The implementation bypasses the gsignal::resample() R wrapper and calls gsignal::upfirdn() (compiled C) directly, with a custom sinc + Kaiser filter for exact MATLAB compatibility. Filter caching across axes provides further speedup in batch processing.

6.2 Scientific Background

Nonlinear gait metrics (LDS, ACI) require fixed-length signals for phase space reconstruction. Since different subjects walk at different cadences, truncated signals have variable lengths. Resampling standardizes these to a common length (18,750 samples) while preserving spectral content up to the Nyquist frequency.

The algorithm operates in 9 steps: GCD reduction of the p/q ratio, sinc + Kaiser filter design, delay computation for phase alignment, zero-padding, polyphase filtering via compiled C (gsignal::upfirdn), and output extraction to the exact target length.

6.3 Function Reference

resample_signal()

Parameter	Type	Default	Description
signal	numeric	(required)	Input signal vector
target_length	integer	(required)	Desired output length
method	character	"auto"	"auto", "direct", or "two_stage"
complexity_threshold	integer	50000	Max filter phases for auto mode
n	integer	20	Filter half-order (ACIER default)
beta	numeric	5	Kaiser window shape parameter
debug	logical	TRUE	Print console diagnostics
use_fallback	logical	TRUE	Fall back to signal/spline if needed
log_file	character	NULL	Path to debug log file

Returns a numeric vector of the target length with metadata attributes: method_used, original_length, filter_phases, filter_length, backend, and elapsed_sec.

Method Selection

Method	Behavior
direct (default in batch)	Single-step polyphase resampling. Works for all ACIER signals.
auto	Falls back to two-stage only when filter_phases exceeds 50,000 (near integer overflow).
two_stage	Stage 1: shrink to 2/3. Stage 2: resample to target. Only when it improves the ratio.

6.4 Debug Logging

The script provides a two-tier logging architecture. Console output shows a compact summary for the first axis only; file logging captures detailed traces for all four axes. The log file contains signal statistics, ratio analysis, filter design details, delay compensation, upfirdn buffer sizes, and output statistics. Enable with:

```
log_path <- "resample_log.txt"
resample_log_header(log_path)     # Once at batch start
resample_signal(signal, target,
  log_file = log_path, log_label = "S01_x")
```

7. Movement Intensity (compute_rms.R)

7.1 Purpose

The compute_rms() function computes two clinically relevant gait metrics: RMS movement intensity (a proxy for walking speed) and the RMS ratio (a speed-independent index of lateral trunk control).

7.2 Scientific Background

Root Mean Square (RMS) of trunk acceleration quantifies the average magnitude of oscillatory movement during walking. Higher RMS values indicate more vigorous gait, typically associated with faster walking speed (Moe-Nilssen, 1998). The RMS ratio (RMS_ML / RMS_norm) addresses speed-dependence by expressing mediolateral acceleration as a proportion of total movement intensity, making it a sensitive marker of balance impairment (Sekine et al., 2013).

In the ACIER study, RMS movement intensity was the strongest discriminator between older and younger adults (Hedges' g = −0.58).

7.3 Function Signature

```
result <- compute_rms(az, norm_signal)
```

Parameter	Type	Description
az	numeric	Mediolateral acceleration in g, after tilt correction
norm_signal	numeric	Vector norm with gravity removed, from RAW signals: sqrt(x^2+y^2+z^2) - 1

7.4 Output

Field	Units	Description	Typical Range
rms_norm	g	RMS of vector norm (movement intensity)	0.15 - 0.60 g
rms_ratio	dimensionless	RMS_ML / RMS_norm (lateral stability)	0.40 - 0.90

7.5 Usage Example

```
result <- compute_rms(
  preprocessed$truncated$z,
  preprocessed$truncated$norm
)
cat("Movement intensity:", result$rms_norm, "g\n")
cat("Lateral stability:", result$rms_ratio, "\n")
```

7.6 Methodological Notes

Norm computation order. The vector norm must be computed from raw (non-tilt-corrected) accelerations because the Euclidean norm is rotation-invariant. Gravity is removed by subtracting 1 g. The tilt-corrected ML signal is used for the ratio numerator.

Difference from Sekine's RMSR. Sekine et al. (2013) define RMSR = RMS_direction / RMS_T using axis-specific RMS without gravity subtraction. The ACIER implementation uses the sample-wise gravity-subtracted norm. Both produce a dimensionless lateral sway index, but absolute values differ. Do not compare RACING output directly with Sekine's published normative values.

8. Gait Regularity (acf_gait_regularity.R)

8.1 Purpose

This script computes step and stride regularity from trunk accelerometer data using the autocorrelation function (ACF). It provides the complete workflow from raw norm signal to publication-ready regularity metrics, including visualization and diagnostic output. The script contains five functions: two low-level helpers (peakn, xcorr_unbiased), one main computation function (compute_gait_regularity), and two user-facing output functions (plot_gait_regularity, print_gait_regularity).

8.2 Scientific Background

The autocorrelation method for trunk accelerometry was introduced by Moe-Nilssen and Helbostad (2004). During walking, the acceleration signal repeats with two characteristic periods: the step period (left-right alternation) and the stride period (full gait cycle). The unbiased ACF of the acceleration vector norm reveals these periodicities as peaks.

Following Auvinet et al. (2002), the Fisher z-transform (atanh) is applied to the raw correlation coefficients. This variance-stabilizing transformation maps bounded values [−1, 1] to an unbounded scale enabling parametric testing. Typical Fisher-transformed values for normal walking range from 1.1 to 1.4 for step regularity and 1.2 to 1.4 for stride regularity.

8.3 Function Reference

compute_gait_regularity()

Parameter	Default	Description
acc_norm	(required)	Gravity-subtracted vector norm, truncated signal at original rate
sampling_freq	256	Sampling frequency in Hz
max_lag_seconds	2.0	Maximum ACF lag in seconds (batch: overridden to 500 samples)
peak_order	60	Peak detection window half-width (234 ms at 256 Hz)
apply_fisher	TRUE	Apply Fisher z-transform (atanh)

Returns a list with 13 fields: step_regularity, stride_regularity (primary outputs), raw correlation values, lag positions in samples, periods in seconds, full ACF vector, detected peaks data frame, and analysis parameters.

plot_gait_regularity()

Creates a publication-quality plot of the ACF with annotated step and stride peaks. Step peak is marked as a green circle, stride peak as red. A legend displays raw and z-transformed values.

print_gait_regularity()

Prints a formatted console summary organized in four sections: step regularity, stride regularity, peak detection summary, and analysis parameters. Returns the result invisibly for chaining.

peakn()

Low-level N-order peak detection helper, ported from MATLAB peakn.m (K. Aminian, EPFL). A point is classified as a peak if and only if it is the strict maximum within a symmetric window of +/-n samples. Peaks are returned in temporal order.

xcorr_unbiased()

Computes unbiased autocorrelation for positive lags (0 to max_lag), replicating MATLAB xcorr(x, maxlag, 'unbiased'). The signal is NOT mean-centered, matching MATLAB behavior. The unbiased estimator r(k) = (1/(n-k)) * sum(x[t]*x[t+k]) prevents artificial ACF decay at large lags.

8.4 Usage Example

```
source("acf_gait_regularity.R")


result <- compute_gait_regularity(
  acc_norm = prep$truncated$norm,
  sampling_freq = 256, peak_order = 60
)
print_gait_regularity(result)
plot_gait_regularity(result, time_axis = TRUE)
```

8.5 Pipeline Integration

In the batch pipeline, gait regularity is computed on the truncated norm signal at the original sampling rate (before resampling). The config controls acf_max_lag (default 500 samples) and peak_min_distance (default 60 samples). Results are written to output columns step_reg and stride_reg.

Important: The norm signal must be computed from RAW (non-tilt-corrected) acceleration as sqrt(ax^2+ay^2+az^2) - 1, making it orientation-independent.

9. Average Mutual Information (ami.R)

9.1 Purpose

This script determines the optimal time delay (tau) for phase space reconstruction, a critical prerequisite for computing LDS and ACI via Rosenstein's algorithm. The script computes Average Mutual Information (AMI) between a signal and its time-delayed copies over a range of lags. The first minimum of the AMI curve identifies the delay providing maximum independent information about the original (Fraser & Swinney, 1986).

The script is a faithful port of four MATLAB functions by Durga Lal Shrestha (2005), deliberately preserving an internal binning inconsistency to ensure numerical compatibility.

9.2 Scientific Background

Phase space reconstruction. Takens' embedding theorem (1981) states that a dynamical system's attractor can be reconstructed from a single scalar time series using time-delayed copies: Y(t) = [x(t), x(t+tau), x(t+2*tau), ..., x(t+(m-1)*tau)]. The quality depends critically on tau.

Why AMI over autocorrelation? The ACF only captures linear dependencies. AMI captures both linear and nonlinear relationships by measuring shared information in an information-theoretic sense. For gait signals, typical optimal delays are 5–20 samples.

The tau convention. The AMI curve has index 1 = lag 0, index 2 = lag 1, etc. When the minimum is at 1-based position k, MATLAB passes k directly as tau. The R implementation preserves this convention throughout the pipeline.

9.3 Function Reference

ami()

Parameter	Type	Default	Description
signal	numeric/matrix	(required)	Univariate time series or 2-column matrix
n_bins	integer	64	Number of bins for probability estimation
n_lags	integer	100	Number of lags to compute
plot	logical	FALSE	Create diagnostic dual-axis plot

Returns a list: ami (values for lags 0 through n_lags), correlation (Pearson), lags (integer vector), optimal_lag (1-based index = MATLAB tau convention).

find_first_minimum()

Finds the first local minimum in the AMI curve. Returns a 1-based position, matching MATLAB convention. Falls back to the 1/e threshold if no local minimum exists; returns NA if no suitable delay is found.

estimate_delay()

Convenience wrapper that returns just the optimal tau value. Defaults to tau = 10 with a warning if no clear minimum is found.

9.4 Binning Strategy

The MATLAB original uses two different binning strategies internally, which are deliberately replicated:

1D marginals (rhist): Center-based bins via seq(min, max, length.out = n_bins), giving bin_width = range/(M-1). Matches MATLAB hist(y, M).

2D joints (prob_2d): Edge-based bins with bin_width = range/M. Matches MATLAB computeEdge() and histc() semantics. With 64 bins, this creates a ~1.6% difference between marginal and joint bin widths.

Adaptive reduction (prob_1d): Bisection search finds the maximum bin count with no empty bins, ensuring valid probability distributions for the MI calculation where log(0) would be undefined.

9.5 Usage Example

```
source("ami.R")


# Standard ACIER pipeline usage
result <- ami(resampled_signal, n_bins = 64, n_lags = 100)
tau <- result$optimal_lag  # e.g., 7


# Pass tau to Rosenstein algorithm
div <- rosenstein_divergence(resampled_signal,
  m = 5, tau = tau, mean_period = 75, max_iter = 1800)


# Quick estimation
tau <- estimate_delay(signal)  # Returns integer


# Diagnostic plot
ami(signal, n_bins = 64, n_lags = 100, plot = TRUE)
```

10. Nonlinear Divergence Analysis

This chapter covers two closely coupled scripts: rosenstein_divergence.R computes the logarithmic divergence curve from reconstructed phase space, and fit_divergence_exponents.R extracts LDS and ACI metrics by fitting linear regressions to specific stride-normalized ranges.

10.1 Rosenstein Divergence (rosenstein_divergence.R)

Purpose

This script computes logarithmic divergence curves using Rosenstein's (1993) algorithm. For each pair of nearest neighbors in reconstructed phase space, it tracks how their Euclidean distance evolves over time, then averages the logarithmic distances at each step (Equation 13 of the paper).

Algorithm

Step	Operation	Description
1	Phase space reconstruction	Build M x m delay embedding matrix using Takens' theorem
2	Nearest neighbor search	For each Y(j), find closest Y(j*) excluding temporal neighbors (Theiler window)
3	Divergence tracking	d(k) = <ln ||Y(j+k) - Y(j*+k)||> averaged over all valid pairs
4	Slope fitting	Handled by fit_divergence_exponents.R

Function: rosenstein_divergence()

Parameter	Type	Default	Description
x	numeric	(required)	Input time series (resampled signal)
m	integer	5	Embedding dimension
tau	integer	10	Time delay in samples (from AMI)
mean_period	integer	NULL	Theiler window; NULL = estimated from ACF
max_iter	integer	1800	Maximum divergence tracking steps
verbose	logical	FALSE	Print progress messages

Returns a list: divergence (average log-divergence for k = 1 to max_iter), time_steps (1:length), and params (m, tau, mean_period, max_iter, N, M).

Supporting Functions

embed_time_series(x, m, tau) — Constructs the M x m delay embedding matrix where M = N - (m-1)*tau.

find_nearest_neighbors(Y, mean_period) — KD-tree nearest neighbor search via RANN::nn2() (O(M log M)), with brute-force fallback (O(M^2)).

compute_divergence(Y, nearest_pos, max_iter) — Vectorized divergence tracking implementing Equation 13.

Performance

For ACIER signals (M ~ 18,700 vectors, max_iter = 1800): with RANN (KD-tree), total computation is 2–5 seconds per axis. Without RANN (brute-force), it rises to 25–45 seconds per axis. RANN is strongly recommended.

10.2 Divergence Exponents (fit_divergence_exponents.R)

Purpose

This script extracts LDS and ACI from Rosenstein divergence curves by fitting linear regressions over stride-normalized time ranges. The x-axis is normalized by stride duration: 1 step = 75 samples, 1 stride = 150 samples.

Fitting Ranges

Metric	Stride Range	Sample Indices	Interpretation
LDS	0 to 0.5 strides	1 to 75	Short-term: dynamic balance within one step
ACI	5 to 12 strides	750 to 1800	Long-term: attractor complexity, gait automaticity

Functions

fit_divergence_exponents(divergence, samples_per_step, lds_steps, aci_stride_range) — Convenience wrapper calling both short-term and long-term fitting. Returns list with LDS, ACI, and params.

fit_short_term_divergence(divergence, samples_per_step, num_steps, return_fit) — LDS extraction. Returns slope in 1/stride units.

fit_long_term_divergence(divergence, samples_per_step, stride_range, return_fit) — ACI extraction. Returns slope in 1/stride units.

plot_divergence_fits(divergence) — Visualization with LDS region (blue) and ACI region (red) highlighted.

10.3 Usage Example

```
source("rosenstein_divergence.R")
source("fit_divergence_exponents.R")

# Compute divergence curve
div <- rosenstein_divergence(signal, m = 5, tau = 17,
  mean_period = 75, max_iter = 1800)

# Extract both exponents
exponents <- fit_divergence_exponents(div$divergence)
cat("LDS:", exponents$LDS, "per stride\n")
cat("ACI:", exponents$ACI, "per stride\n")
```

```
# Visualization
plot_divergence_fits(div$divergence)

# Detailed analysis with R-squared
detailed <- fit_divergence_exponents(
  div$divergence, return_fits = TRUE)
cat("LDS R^2:", detailed$fits$LDS$r_squared, "\n")
```

10.4 Typical Value Ranges

For healthy older adults walking normally (ACIER data): LDS-ML ~ 0.8–1.5 per stride, ACI-Norm ~ 0.02–0.04 per stride, ACI-AP ~ 0.01–0.04 per stride. ACI values can be negative (attractor contraction), which is valid. For well-behaved signals, LDS R-squared should exceed 0.95; ACI R-squared is typically 0.85–0.98.

11. Batch Processing Pipeline

This chapter documents the four scripts forming Layer 3 of the RACING pipeline: the automated batch processing system that enables researchers to process entire datasets with a single command.

11.1 Configuration (read_config.R)

read_racing_config()

Reads all parameters from the 1_Configuration sheet of a RACING config Excel file, validates them, computes derived values, and returns a named list.

Parameter Group	Keys	Source Rows
Input Data	data_folder, sampling_freq, col_ap/v/ml, accel_units, csv_header, csv_delim	8-15
Preprocessing	apply_tilt, auto_orient	20-21
Signal Processing	target_steps, samples_per_step, target_samples (computed)	26-28
SF Detection	sf_freq_min, sf_freq_max	33-34
Linear Metrics	acf_max_lag, peak_min_distance	39-40
AMI / Phase Space	ami_n_bins, ami_n_lags, embedding_dim	45-47
Divergence	divergence_length, mean_period, lds_range_end, aci_range_start/end	52-56
Output	output_folder, study_name, export_div_curves, export_resampled	60-63

The validate_racing_config() function checks for missing parameters, invalid types, out-of-range values, and logical consistency. The print_config_summary() function displays a formatted ASCII box of all parameters.

11.2 Data Import (import_csv_lowback.R)

import_csv_lowback()

Imports a single CSV file containing tri-axial accelerometer data using configuration parameters. Features automatic comment line detection (supports datasets like LTMM with # headers), fast import via data.table::fread() with base R fallback, and unit conversion from m/s2 to g when needed.

Parameter	Type	Description
file_path	character	Full path to the CSV file
config	list	Configuration from read_racing_config()
verbose	logical	Print diagnostic messages [FALSE]

Returns a data.frame with 3 columns (ap, v, ml) in g units, with attributes: sampling_rate, units, file, n_samples, import_warnings, and col_indices.

11.3 Pipeline Orchestration (run_gait_analysis_batch.R)

The master script that reads configuration, discovers CSV files, and processes each through the complete analysis pipeline. For each file, it executes:

Step	Operation	Script
A	CSV import	import_csv_lowback.R
B	Tilt + SF + Truncate + Resample	truncate_resample.R
D	RMS (norm and ratio)	compute_rms.R
E	ACF step/stride regularity	acf_gait_regularity.R
F	AMI (4 axes: AP, V, ML, norm)	ami.R
G	Rosenstein + LDS/ACI (4 axes)	rosenstein_divergence.R + fit_divergence_exponents.R
H	Optional: export divergence curves	write.csv
I	Optional: export resampled signals	write.csv

Each axis (AP/x, V/y, ML/z, norm) gets its own AMI-determined time delay (tau) for phase space reconstruction. All calls are wrapped in tryCatch for graceful error handling.

11.4 Output (write_results.R)

Formats and saves results to a multi-sheet Excel workbook. The output contains 21 columns per file:

Column	Metric	Type
A	file_id (sequential)	Integer
B	subject_id (filename)	String
C	n_steps (actual)	Numeric
D	SF_hz	Numeric
E	RMS_norm	Numeric
F	RMS_ratio_ml_norm	Numeric
G-H	step_reg, stride_reg	Numeric
I-L	AMI_x, AMI_y, AMI_z, AMI_norm	Integer
M-P	LDS_x, LDS_y, LDS_z, LDS_norm	Numeric
Q-T	ACI_x, ACI_y, ACI_z, ACI_norm	Numeric
U	warnings	String

11.5 Complete Batch Example

```
# Source all scripts
source("read_config.R")
source("import_csv_lowback.R")
source("tilt_correction.R")
source("step_frequency_detection.R")
```

```
source("resample_signal.R")
source("truncate_resample.R")
source("compute_rms.R")
source("acf_gait_regularity.R")
source("ami.R")
source("rosenstein_divergence.R")
source("fit_divergence_exponents.R")
source("write_results.R")
source("run_gait_analysis_batch.R")

# Run the pipeline
config <- read_racing_config("RACING_config.xlsx")
results <- run_gait_analysis_batch(config)
```

Appendix A: Quick Reference Card

Essential Constants

Constant	Value	Description
SAMPLING_FREQ	256 Hz	Physilog sensor sampling rate
TARGET_STEPS	250	Standardized step count (indoor)
SAMPLES_PER_STEP	75	Samples per step after resampling
TARGET_SAMPLES	18,750	250 steps x 75 samples/step
EMBEDDING_DIM	5	Phase space dimensions
LDS_RANGE	0-75 samples	Short-term divergence (0-0.5 strides)
ACI_RANGE	750-1800 samples	Long-term divergence (5-12 strides)
AMI_BINS	64	Default histogram bins for AMI
AMI_LAGS	100	Maximum AMI search range
THEILER_WINDOW	75 samples	Temporal exclusion in NN search

Pipeline Function Calls (Minimal)

```
# 1. Preprocessing
prep <- truncate_resample(acc_x, acc_y, acc_z)

# 2. Linear metrics (on truncated signals)
rms  <- compute_rms(prep$truncated$z, prep$truncated$norm)
reg  <- compute_gait_regularity(prep$truncated$norm)

# 3. Nonlinear metrics (on resampled signals, per axis)
ami_res <- ami(prep$resampled$norm, n_bins=64, n_lags=100)
div     <- rosenstein_divergence(prep$resampled$norm,
             m=5, tau=ami_res$optimal_lag,
             mean_period=75, max_iter=1800)
exp     <- fit_divergence_exponents(div$divergence)
```

Appendix B: Glossary

Term	Definition
ACI	Attractor Complexity Index: long-term divergence slope (5-12 strides) measuring gait automaticity
ACF	Autocorrelation Function: measures signal self-similarity at different lags
ACIER	Attractor Complexity Index Empirical Rationalization: the source study (2021-2024)
AMI	Average Mutual Information: information-theoretic measure for optimal embedding delay
AP	Anteroposterior: the forward-backward axis of movement
DFA	Detrended Fluctuation Analysis: method for quantifying long-range correlations
Embedding dimension	Number of coordinates in reconstructed phase space (m = 5 in ACIER)
Fisher z-transform	atanh(r): variance-stabilizing transform for correlation coefficients
KD-tree	k-dimensional tree: data structure for efficient nearest-neighbor search
LDS	Local Dynamic Stability: short-term divergence rate (0-0.5 strides) measuring balance
LTMM	Long-Term Movement Monitoring: PhysioNet free-living accelerometer database
ML	Mediolateral: the side-to-side axis of movement
Phase space	Multidimensional space reconstructed from time-delayed copies of a signal
Polyphase filter	FIR filter implementation that combines upsampling, filtering, and downsampling
RACING	R-based Attractor Complexity INdex for Gait
RMS	Root Mean Square: average signal magnitude, proxy for walking speed
Rosenstein algorithm	Method for estimating largest Lyapunov exponent from small datasets (1993)
Stride	One complete gait cycle (two steps): left heel-strike to left heel-strike
Step	Half gait cycle: left heel-strike to right heel-strike (or vice versa)
Takens' theorem	Mathematical basis for reconstructing dynamical attractors from scalar time series
Tau (tau)	Time delay for phase space embedding, determined by AMI first minimum
Theiler window	Temporal exclusion zone in nearest-neighbor search (prevents autocorrelation artifacts)
V	Vertical axis of movement

Appendix C: References

[1] Auvinet, B., Berrut, G., Touzard, C., et al. (2002). Reference data for normal subjects obtained with an accelerometric device. Gait & Posture, 16(2), 124-134.

[2] Dingwell, J. B., & Cusumano, J. P. (2000). Nonlinear time series analysis of normal and pathological human walking. Chaos, 10(4), 848-863.

[3] Fraser, A. M., & Swinney, H. L. (1986). Independent coordinates for strange attractors from mutual information. Physical Review A, 33(2), 1134-1140.

[4] Gasior, M., & Gonzalez, J. L. (2004). Improving FFT frequency measurement resolution by parabolic and Gaussian interpolation. AIP Conference Proceedings, 732(1), 276-285.

[5] Moe-Nilssen, R. (1998). A new method for evaluating motor control in gait under real-life environmental conditions. Part 1: The instrument. Clinical Biomechanics, 13(4-5), 320-327.

[6] Moe-Nilssen, R., & Helbostad, J. L. (2004). Estimation of gait cycle characteristics by trunk accelerometry. Journal of Biomechanics, 37(1), 121-126.

[7] Piergiovanni, S., & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784.

[8] Piergiovanni, S., & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

[9] Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. Physica D, 65(1-2), 117-134.

[10] Sekine, M., et al. (2013). A gait abnormality measure based on root mean square of trunk acceleration. Journal of NeuroEngineering and Rehabilitation, 10, 118.

[11] Takens, F. (1981). Detecting strange attractors in turbulence. In Dynamical Systems and Turbulence, Lecture Notes in Mathematics 898, 366-381. Springer.

[12] Terrier, P., & Reynard, F. (2018). Maximum Lyapunov exponent revisited: Long-term attractor divergence of gait dynamics is highly sensitive to the noise structure of stride intervals. Gait & Posture, 66, 236-241.

[13] Theiler, J. (1986). Spurious dimension from correlation algorithms applied to limited time-series data. Physical Review A, 34(3), 2427-2432.

RACING User Guide

Version 1.0 — March 2026

RACING is open-source software developed under the ORD 2025 grant.
Source code: github.com (RACING repository)
ACIER dataset: Zenodo DOI 10.5281/zenodo.10148824
Reproducibility capsule: CodeOcean

HE-Arc Santé — HES-SO University of Applied Sciences and Arts Western Switzerland